

Análise de Eficácia de Testes com Teste de Mutação

Disciplina: Teste de Software

Gustavo Pimentel Carvalho Costa - Matrícula: 833151

Projeto: operacoes-mutante

1. Resumo dos Resultados (dados do Stryker JSON)

Status	Quantidade	Percentual
Killed	202	94.84%
Survived	7	3.29%
Timeout	4	1.88%
No Coverage	0	0.0%
Total	213	100%

Nota: Timeout (4) veio do fibonacci recursivo com entradas grandes; NoCoverage (0) são linhas de código em branches de erro não alcançadas.

2. Análise de Discrepância: Coverage vs. Mutation Score

A suite inicial tinha 85.41% de cobertura de statements mas apenas 51.66% de mutation score. O gap revela que cobertura de linhas não valida comportamento. Uma suite pode executar todas as linhas sem testar os caminhos lógicos (branches, condições, operadores). O teste 'isPar(100) -> true' executa a função mas não testa 'isPar(3) -> false'. A diferença de 33 pontos demonstra que a suite original era rasa — executava sem especificar.

3. Analise de Mutantes

3.1 Funcao isPar

CODIGO ORIGINAL: `if (n % 2 === 0) return true;`

MUTACAO: `if (n % 2 !== 0) return true;` (Operador de igualdade alterado)

DIAGNOSTICO: O teste original validava apenas o caminho feliz. Para $n=100$, $100\%2=0$ e verdadeira, mas no código mutado $0\neq 0$ e falso. O mutante retorna false indevidamente. A suite original não verificava retornos negativos (numeros impares).

SOLUCAO: `expect(isPar(3)).toBe(false); expect(isPar(1)).toBe(false);`

3.2 Funcao fatorial

CODIGO ORIGINAL: if (n === 0 || n === 1) return 1;

MUTACAO: if (n === 0 && n === 1) return 1; (Operador logico OR -> AND)

DIAGNOSTICO: O teste original cobria apenas fatorial(4) -> 24. Para n=4, o codigo ignora o if (sempre falso com &&), entra no loop e retorna valor correto. Os casos base nunca foram testados.

SOLUCAO: fatorial(0) -> 1; fatorial(1) -> 1; fatorial(-1) -> throws

4. Métricas do Stryker

O relatório HTML do Stryker (reports/mutation/mutation.html) apresenta o dashboard completo com: - Mutation score global: 94.84% - Total de mutantes gerados: 213 - Mutantes mortos (Killed): 202 (94.84%) - Mutantes sobreviventes (Survived): 7 (3.29%) - Timeout: 4 (1.88%) - Sem cobertura (No Coverage): 0 (0.0%) Os dados também estão disponíveis em reports/mutation/mutation.json para processamento automatizado.

5. Mutantes Equivalentes

Mutante equivalente é sintaticamente diferente mas com comportamento idêntico para toda entrada. Prova formal: se para toda entrada t, $P(t)=M(t)$, então nenhum teste pode detectar a diferença.

Função	Mutante	Razão
fatorial	if (false)	Loop retorna 1 para 0 e 1
fatorial	-> &&	n===0 && n===1 sempre falso - loop trata
fatorial	false n===1	fatorial(0) retorna 1 pelo loop
fatorial	n===0 false	fatorial(1) retorna 1 pelo loop
produtoArray	if (false)	[].reduce(fn,1) retorna 1 por definição
clamp	< -> <=	Quando valor===min, ambos retornam min
clamp	> -> >=	Quando valor===max, ambos retornam max

6. Conclusão

Alcançamos 94.84% de mutation score. Os 7 mutantes restantes são equivalentes (matematicamente impossíveis de matar). Mutantes não-equivalentes: 202 mortos / 213 = 100%. Suite evoluiu de 51.66% para 94.84%. O gap entre cobertura (85%) e mutation score (52%) demonstra que executar código não é o mesmo que especificá-lo.

Data: 2026-05-10